



CYART

inquiry@cyart.io

www.cyart.io

AI Context Project

Phase 1 – Week 2 Technical Documentation

Prepared by
AI Context Squad

Date:
17 August 2026



Table of Contents

No.	Section
1.	Introduction
2.	Objective
3.	Technology stack
4.	RAG Pipeline Overview
5.	Asset Data Model Implementation
6.	Embedding Generation
7.	Vector Index (HNSW)
8.	Asset Search Endpoint
9.	Asset Summary Prompt Template
10.	Accuracy Check
11.	Testing and Verification
12.	Issues Encountered and Resolutions
13.	Conclusion



1. Introduction

The AI Context project deals with the development of an intelligent asset search engine that is a part of the AEV Platform. The intelligent asset search engine is intended for processing asset data and making it searchable with the help of AI techniques.

In Week 2, the AI Context Squad moved from the backend base built in Week 1 to the implementation of the RAG pipeline's first version.

The Week 2 implementation was based on preparing searchable asset data, creating embeddings, setting up vector similarity search, building asset search endpoint, making asset summary prompt template, and checking search relevance.

2. Objective

The key aim of Week 2 was to create the initial prototype of the RAG pipeline for the AI Context service.

Key goals included:

- Creating the storage schema for searchable asset data.
- Create embeddings for sample asset data.
- Creating a vector index for efficient search by similarity.
- Creating the /copilot/asset-search endpoint.
- Defining the initial “asset summary” prompt template.
- Conducting an accuracy check via sample queries.
- Providing documentation for the search endpoint and prompt template.



3. Technology Stack

Technology	Purpose
Python	Backend programming language
FastAPI	REST API framework
Uvicorn	ASGI server
PostgreSQL	Relational database
pgAdmin 4	Database administration
Visual Studio Code	Development environment
pgvector	Stores vector embeddings

4. RAG Pipeline Overview

The retrieval-augmented generation (RAG) is the strategy employed in the AI Context project to fetch the asset information and make use of the fetched information as the context for generation of the AI response.

In Week 2, the AI Context Squad has been working on the first iteration of the RAG pipeline, which involves fetching the searchable asset information, creating vector embeddings out of it, indexing the vectors and making the retrieval functions accessible through the asset search endpoint. The asset information thus fetched can then be provided to the “asset summary” prompt template.



Overall flow could be represented as:

Asset Data → Embedding Generation → Vector Index → Similarity Search → Asset Search Endpoint → Asset Summary Prompt → AI Response

The functions of each stage include the following:

- Asset Data – contains information regarding the security assets.
- Embedding Generation – generates vectors from asset information.
- Vector Index – allows efficient searching for similar assets.
- Asset Search Endpoint – receives the search request and finds relevant asset information.
- Asset Summary Prompt – provides the instruction on how to generate a useful asset summary from found information.
- AI Response – gives the final answer based on the asset context.

5. Asset Data Model Implementation

Data schema is responsible for the structure of the asset information storage in the AI Context service. Asset model includes all the fields necessary for asset identification, description, risk information, and embedding generation for the purpose of similarity search.

The asset data is stored in PostgreSQL, whereas the embedding field contains the vectors created from asset information.



id	Integer	Unique identifier for the asset
asset_name	String	Name of the asset
asset_type	String	Type/category of the asset
operating_system	String	Operating system associated with the asset
ip_address	String	IP address of the asset
risk_score	Integer	Risk score associated with the asset
description	Text	Additional information about the asset
embedding	Vector(384)	Vector representation used for similarity search
created_at	DateTime	Timestamp for when the asset record was created

5.1 Asset Model Implementation

```
se X models.py test_isolation.py expiry_job.py PRIVACY.md embedding.py generate_asset_embeddings.py requirements.txt AI Context Project
app > models.py
35
36
37 class Asset(Base):
38     __tablename__ = "assets"
39
40     id = Column(Integer, primary_key=True, index=True)
41     asset_name = Column(String(255), nullable=False)
42     asset_type = Column(String(100), nullable=True)
43     operating_system = Column(String(100), nullable=True)
44     ip_address = Column(String(50), nullable=True)
45     risk_score = Column(Integer, nullable=True)
46     description = Column(Text, nullable=True)
47     embedding = Column(Vector(384), nullable=True)
48
49     created_at = Column(
50         DateTime(timezone=True),
51         server_default=func.now()
52     )
53
```



6. EMBEDDINGS GENERATION

Embeddings are numeric vector representations of textual data. It allows comparison of the asset data using semantic similarities rather than exact keyword matches only.

The name of the embedding model that was implemented is all-MiniLM-L6-v2. After generating the embedding, it is transformed into a list and saved in the asset embedding field of the database.

The process of generating embeddings may be summarized as:

Asset Information → Text Representation → Embedding Model → Vector
Embedding → Database

Embedding Generation Process

The generate_asset_embeddings.py script extracts the records of assets from the database, transforms them into text representations, generates their embeddings, and stores them back in the database.

```
se.py  models.py  test_isolation.py  expiry_job.py  PRIVACY.md  embedding.py  generate_asset_embeddings.py  requirements.txt  AI Context Project.d...
app > embedding.py
1  from sentence_transformers import SentenceTransformer
2
3  model = SentenceTransformer("all-MiniLM-L6-v2")
4
5
6  def generate_embedding(text: str):
7      embedding = model.encode(text)
8      return embedding.tolist()
```



7. VECTOR INDEX (HNSW)

The vector index is used to enhance the efficiency of similarity search in the stored embeddings. The Week 2 RAG model has been implemented with an HNSW vector index.

HNSW has been designed for efficient approximate nearest-neighbor search. With HNSW, the system would be able to find out the asset embeddings which are the most similar to the embedding created from the search query.

Thus, vector search would be able to find assets on the basis of the meaning of the search query instead of the exact match.

The screenshot shows a database interface with a dark theme. At the top, there's a header with 'Untitled', a 'Save' button, and a dropdown menu showing 'Primary' and 'Active'. Below this, there's a dropdown for 'neondb'. The main area contains two SQL queries. The first query is a 'CREATE INDEX' statement for 'assets_embedding_hnsw_idx' on the 'assets' table using the 'hnsw' method. The second query is a 'SELECT' statement to retrieve the index name from the 'pg_indexes' table. Below the queries, there's a status bar indicating 'Connected (2 queries)'. There are buttons for 'Run', 'Explain', and 'Analyze'. To the right of these buttons, it shows '296ms' and '2 rows'. Below this, there's a table with two columns: '#', 'indexname'. The table contains two rows: '1 assets_embedding_hnsw_idx' and '2 assets_pkey'. At the bottom right, there are icons for copy, download, and print.

```
1 CREATE INDEX IF NOT EXISTS assets_embedding_hnsw_idx
2 ON assets
3 USING hnsw (embedding vector_cosine_ops);
4
5 SELECT indexname
6 FROM pg_indexes
7 WHERE tablename = 'assets';
```

Connected (2 queries)

Run Explain Analyze 296ms 2 rows

#	indexname
1	assets_embedding_hnsw_idx
2	assets_pkey



8. ASSET SEARCH ENDPOINT

The asset search endpoint gives the API functionality that allows you to find suitable assets based on a given search query. This endpoint is a crucial element of the RAG pipeline since it links the query of the user with the process of finding assets using vectors.

The asset search endpoint:

/copilot/asset-search

The flow of this endpoint is:

User Query → Query Embedding → Vector Similarity Search → Relevant Assets → Search Response

The assets obtained in this way can be further used in the asset summary prompt.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8001/copilot/asset-search' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "query": "find high risk windows assets"
  }'
```

Request URL

http://localhost:8001/copilot/asset-search

Server response

Code Details

200

Response body

```
{
  "query": "find high risk windows assets",
  "total_results": 3,
  "results": [
    {
      "id": 1,
      "asset_name": "Windows Server",
      "asset_type": "Server",
      "operating_system": "Windows Server 2022",
      "ip_address": "192.168.1.20",
      "risk_score": 80,
      "description": "Finance application server",
      "similarity_score": 0.4183
    },
    {
      "id": 4,
      "asset_name": "Azure VM",
      "asset_type": "Virtual Machine",
      "operating_system": "Windows Server 2019",
      "ip_address": "192.168.1.20",
      "risk_score": 81,
      "description": "Internal application server",
      "similarity_score": 0.3606
    },
    {
      "id": 3,
      "asset_name": "AWS EC2",
      "asset_type": "Cloud Instance",
      "operating_system": "Amazon Linux",
      "ip_address": "192.168.1.20",
      "risk_score": 82,
      "description": "Internal application server",
      "similarity_score": 0.3506
    }
  ]
}
```

Download



9. ASSET SUMMARY PROMPT TEMPLATE

The Asset Summary Prompt Template is utilized to give structured directions on how to create an asset summary from retrieved asset information.

The prompt gives relevant information from assets that have been retrieved from the search process and informs the AI model how to understand the information and summarize it.

The role of the prompt template is to ensure that there is consistency in the format of the response and that the response created will be based on the retrieved asset information.

Prompt Template Structure: There are three main parts that constitute the asset summary prompt template.,

1. Context - This gives the information from the retrieved assets to the model.
2. Instruction – This defines what should be done to the retrieved asset information.
3. Output Requirement – This specifies how the response should be created.

```
1 SELECT column_name
2 FROM information_schema.columns
3 WHERE table_name = 'prompt_templates'
4 ORDER BY ordinal_position;
```

Connected (1 query)

Run Explain Analyze 370ms 6 rows

#	column_name
1	id
2	template_name
3	version
4	template_content
5	is_active
6	created_at



10. TESTING AND VERIFICATION

Testing was conducted when implementing the Week 2 RAG pipeline in order to check whether individual components work as expected.

This involved:

- Verification of the asset database schema.
- Verification of successful asset embeddings generation.
- Confirmation that generated asset embeddings can be saved to the database.
- Asset search endpoint testing.
- Natural language search queries testing.
- Relevance of the retrieved assets verification.
- Asset summary prompt verification.

The screenshot displays a web browser window with a REST client interface. The 'Curl' tab is active, showing a cURL command for a POST request to `http://localhost:8001/copilot/asset-search`. The request headers include `accept: application/json` and `Content-Type: application/json`. The request body is a JSON object with a query and a limit. The 'Server response' section shows a 200 status code and a detailed JSON response body. The response body contains a query, total results, and a list of three assets with their details. The 'Response headers' section shows the content length, content type, date, and server information.

```
curl -X 'POST' \
  'http://localhost:8001/copilot/asset-search' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "query": "public facing cloud instance",
    "limit": 3
  }'
```

Request URL

`http://localhost:8001/copilot/asset-search`

Server response

Code Details

200

Response body

```
{
  "query": "public facing cloud instance",
  "total_results": 3,
  "results": [
    {
      "id": 3,
      "asset_name": "AWS EC2",
      "asset_type": "Cloud Instance",
      "operating_system": "Amazon Linux",
      "ip_address": "10.0.0.15",
      "risk_score": 90,
      "description": "Public-facing EC2 instance",
      "similarity_score": 0.5634
    },
    {
      "id": 2,
      "asset_name": "Azure VM",
      "asset_type": "Virtual Machine",
      "operating_system": "Windows Server 2019",
      "ip_address": "10.10.1.25",
      "risk_score": 81,
      "description": "Internal application server",
      "similarity_score": 0.2968
    },
    {
      "id": 1,
      "asset_name": "Ubuntu Server",
      "asset_type": "server",
      "operating_system": "Ubuntu 22.04",
      "risk_score": 0,
      "description": "Internal application server",
      "similarity_score": 0.0
    }
  ]
}
```

Response headers

```
content-length: 694
content-type: application/json
date: Tue, 10 Aug 2026 09:37:21 GMT
server: uvicorn
```

Responses



11. ISSUES ENCOUNTERED AND RESOLUTIONS

While implementing the software, some configuration and dependencies issues were faced. These issues were sorted out during the development and testing phase.

Issues Faced and Solutions

1. Docker configuration problem: Initially the Docker command was not recognized and Docker Compose started facing issues related to startup and build. This problem was solved using proper Docker configuration and rebuilding the Docker container.
2. Pyantic Field Problem: An error named `NameError: name 'Field' is not defined` was faced. This issue was solved by importing `Field` from `Pyantic`.
3. Python indentation and import problem: Indentation and model import problems were faced while adding functionality to APIs. This problem was fixed in their respective Python files.
4. Vector Search Verification: After configuration of `pgvector/HNSW` and embedding, vector search worked fine and gave ranked results with similarity score in Neon.



12. CONCLUSION

The second week of the project was dedicated to developing the first RAG pipeline for the AI Context service. The RAG pipeline consisted of structured asset data, embedding generation, vector similarity search, and asset search endpoint.

Asset Summary Prompt Template was also generated to provide a framework for building a summary based on retrieved asset data. The testing and validation have been done to ensure the functionality and relevancy of the search process.

The results obtained in the second week will serve as a basis for further enhancement of the AI Context service in terms of prompts, search process and the whole AEV Platform.

Moreover, the implementation provided in the second week gives a practical basis for semantic asset retrieval using the AI Context service. The combination of vector embeddings, vector similarity search, and prompt templates proves that the asset information could be effectively retrieved and provided in a form of context for AI-assisted response generation. The implementation could be further expanded in the following weeks.